

Use Case Diagram

د. ماهر الصارم



17/10/2023

RB Informatics;

هندسة البرمجيات (1)

Use Case diagram مخطط حالات الاستخدام

مخطط يستخدم لوصف حالة وإجراءات النظام ومن يمثل أو يتعامل مع هذا الإجراء من الخارج بشكل مباشر، أي يقوم بتحديد الوظائف وكيفية تفاعل النظام والمستخدمين معها .

خلاصة:

■ Use Case diagram: هو مخطط لا يتعلق بكيفية عمل النظام أو مكوناته بل يعطي لمحة عامة عن النظام دون ذكر تفاصيل كل حالة.

كيف يتم إنشاء مخطط حالات الاستخدام :

في بعض الأحيان يكون النظام المطلوب مصاغ بشكل سردي، عندها يتم صياغة وثيقة متطلبات النظام SRS (دفتر الشروط) بشكل بنود، نقرأه ونستخرج المصادر والأفعال، إذ أن كل فعل أو مصدر يعبر عن حالة استخدام (وظيفة في النظام) ضمن مخطط حالات الاستخدام الذي سنقوم بوضعه، ثم نضع جدول مكون من عامودين الأول عبارة عن حالة الاستخدام "use case" والتي تكافئ الوظيفة، والثاني عبارة عن الفاعل "actor" وهو الجهة التي ستقوم بتنفيذ الوظيفة .

تكون وثيقة المتطلبات مكتوبة على الشكل التالي "على سبيل المثال " :

- يجب على النظام
- يميز النظام بين أربع أنواع من
- يجب أن يتيح النظام
- يسمح النظام ب



مثال : ترغب مكتبة بأتمتة أعمالها بحيث تتيح للطلاب والمدرسين حجز كتب ... إلخ

نقوم بداية بصياغة وثيقة المتطلبات التي تبيين حدود النظام "scope" وهي الوظائف التي يقوم بها النظام وذلك

بشكل بنود لسهولة وضع مخطط حالات الاستخدام، مثلاً :

1. يجب على النظام أن يسمح للطلاب والمدرس بحجز كتاب قبل أن يتم تسجيل استعارته .
 2. يجب أن يسمح النظام للطلاب بإلغاء حجز كتاب عندما يتم إعادته .
 3. يسمح النظام بعملية تسجيل استعارة لطلاب قام بحجز كتاب عن طريق أمين المكتبة " مثلاً " .
 4. يجب على النظام أن يسمح بعملية تسجيل إعادة كتاب عن طريق أمين المكتبة .
- ثم نقوم برسم الجدول التالي " Use Case list " :

Use Case Id	Actor	Use Case
UC-1	طالب / مدرس	حجز كتاب
UC-2	طالب / مدرس	إلغاء حجز كتاب
UC-3	أمين المكتبة	تسجيل استعارة كتاب
UC-4	أمين المكتبة	تسجيل إعادة كتاب

ثم نقوم برسم مخطط حالات الاستخدام بعد التعرف على محددات المخطط وبعد ذلك نقوم برسم جدول يدعى Use Case specification يحوي تفاصيل إضافية لا يتم ذكرها في المخطط .

1. Note : حتى نقوم برسم مخططات الـ UML نستطيع استخدام إحدى هذه الأدوات "Tools"

- Star UML
- Draw.io
- Enterprise after effect
- Visio

هذه الأدوات تساعدنا لإنشاء مخططات UML، فهي أدوات تساعد المهندس في عملية تطوير الأنظمة وإنشاء المشاريع بـ CASE Tools وهو اختصار لـ : Computer-Aided Software Engineer .

2. لنعرف أكثر عن الـ SRS وكيفية كتابتها نستطيع البحث في الإنترنت والمراجع وتوثيل قوالب عنها "IEEE SRS Template" تحتوي على عناوين وبدخلها شرح عن ماذا يجب أن نكتب فيها .

- Use Case diagram : يُوصف (ماذا يعمل النظام ؟).
- Activity diagram : يُوصف (آلية عمل النظام " كيف يعمل النظام ؟ ").
- Class diagram : يُوصف (مما يتركب النظام برمجياً).

٤ محددات مخطط حالات الاستخدام Use Case diagram :

لكل نظام معلومات مجموعة من المحددات "أشكال معينة لرموز معينة" ويتألف مخطط Use case diagram من المحددات التالية :

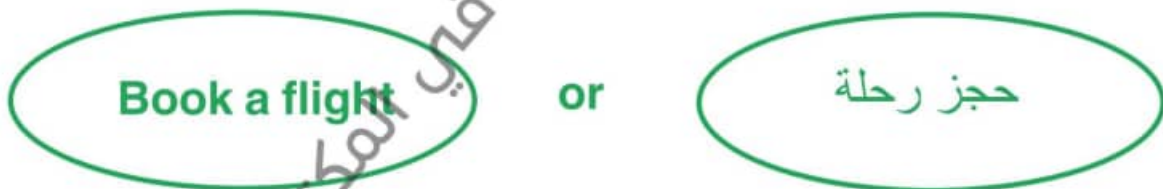
Use case_ Actors_ System boundary _ Relationships

Use Cases

حالة الاستخدام هي مجموعة من تسلسلات الإجراءات "Flow of events (activities)" التي يقوم بها النظام والتي تسفر عن نتيجة ملحوظة ذات قيمة إلى جهة فاعلة معينة، وتعتبر عن وظيفة يقوم بها الـ Actor داخل النظام أو ملخص لسيناريوهات لتحقيق أو عدم تحقيق مهمة معينة. وتصف متطلباً للنظام .

وتمثل حالة الاستخدام بشكل بيضوي "قطع ناقص" حيث نضع اسمها بداخلها تعبيراً عنها ويكون اسمها مصدراً في حال كان الاسم مكتوباً باللغة العربية وفعل في حال كان مكتوباً باللغة الإنكليزية ويعبر عن وظيفة ما بشكل عام وليس تفصيلاً، ويكون مكون من ثلاث أو أربع كلمات فقط ويفضل أن يكون اسمها باللغة الإنكليزية .

مثال 1 : شخص قادر على حجز "Book" لرحلة معينة :



مثال 2 : نظام بنكي يتيح للمستخدم الاستعلام عن رصيده، تحويل رصيد ودفع فواتير :



- كل وظيفة في وثيقة المتطلبات SRS نحولها لـ Use Case .



يمكن أن نتعرف على حالات الاستخدام عبر مجموعة من الأسئلة المطروحة :

- ما هي المهام التي يحتاجها المتفاعلون "Actors" من النظام ؟
- هل يقوم النظام بتخزين المعلومات ؟ إن كان الجواب نعم ...
- فهل يوجد متفاعلون يقومون بإجراء عمليات CRUD على المعلومات ؟

🚩 CRUD : Create_ Read_ Update_ Delete

System boundary

هي الحدود التي يكون ضمنها النظام ويتم التعبير عن حدود النظام بمستطيل يتم وضع الحالات (Use cases) ضمنه والمتفاعلين (Actors) خارجه.

نستطيع وضع المهام التي يقوم بها النظام ضمن وثيقة المتطلبات تحت عنوان إطار النظام ووضع المهام التي لا يقدمها النظام تحت العنوان خارج إطار النظام وذلك في حال وجود مجموعة من المهام التي لا يحققها النظام .
مثال : أتمتة ذاتية الموظف دون الاهتمام بحساب الراتب والأمور المالية، فنكتب كالتالي :
الإطار : تسجيل إجازة - منح إجازة صحية .

خارج الإطار / لا يهتم النظام بالأمور المالية المتعلقة بالموظف

وبالتالي ضمن الـ **System boundary** تتواجد جميع الوظائف الموصوفة ضمن الإطار .

Actors

الفاعل هو شخص أو شيء يتفاعل مع النظام، هو من أو ما يتعامل مع النظام بشكل مباشر دون وسيط، اذ يتواصل الممثل مع النظام عن طريق إرسال واستقبال الرسائل، فالفاعل هو الدور الذي يلعبه المستخدم فيما يتعلق بالنظام.

وله الشكل التالي حيث يتم كتابة اسمه تحته ويكون اسمه اسم فاعل :

يمكن أن نتعرف على الـ actors من خلال طرح مجموعة من الأسئلة :

من يستخدم النظام؟ من يحصل على المعلومات من النظام ؟

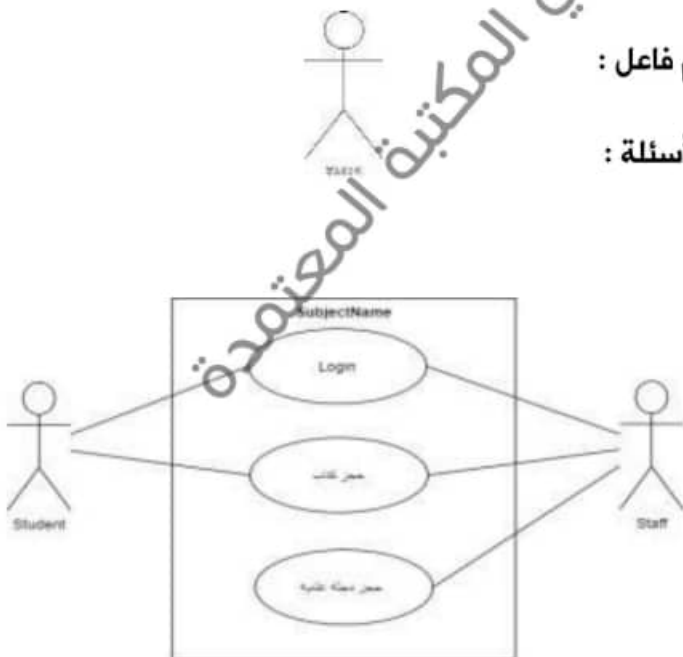
من يزود النظام بالمعلومات ؟

ما هي الأنظمة الأخرى التي يستخدمها هذا النظام ؟

من يقوم بتثبيت النظام أو بدء تشغيله أو صيانته ؟

هل هنالك وظائف تتفعل بشكل أوتوماتيكي تبعاً للمكان أو

الوقت أو تغير حالة ما ؟



هل يوجد أحداث ممكنة الوقوع يجب أن يعرفها النظام ؟ (مثل نظام ذكي في مركز ما يتضمن النظام وظيفة التحكم بالتكييف في المركز، فيجب أن يقوم النظام بتعديل التكييف عند حدوث عاصفة شمسية على سبيل المثال) .

ويمكن أن يكون الـ actor :

- شخص أو طالب أو مدير أو " حسب طبيعة النظام " .
- جهاز " مثل scanning machine " يعتمد فكرة الحساس مثل جهاز barcode، وذلك عندما يقوم بإجراء الوظيفة بشكل تلقائي دون وجود شخص ما يقوم باستخدامه.
- **Timer** يتم ضبطه على موعد محدد فيقوم بتنفيذ مهمة معينة كلما حان الوقت، مثل مهمة إرسال إنذار للمتأخرين عن تسديد فاتورة الهاتف " وبالتالي عندما تصادفنا حالة مثل هذه أي لا يوجد متفاعل حقيقي يقوم بإنجاز الوظيفة لا يجوز أن نضع الـ actor هو النظام بل نضع الـ actor هو trigger يكون مسؤول عن إنجاز المهمة " .

■ توضيح : مثلاً في حال مصادفة بند في وثيقة المتطلبات مذكور بالشكل التالي: يجب على النظام أن يقوم بفرز الطلاب وفق الترتيب الأبجدي عند انتهاء فترة التسجيل، نلاحظ في هذه الحالة أن النظام هو من ينجز المهمة ولكن لا نضع النظام ك متفاعل بل نضع الـ timer ك متفاعل .

■ عندما يكون عدد حالات الاستخدام والمتفاعلين كبير، نقوم عندها بإنشاء subsystems داخل النظام وذلك بتجميع حالات الاستخدام المرتبطة مع بعضها ضمن نظام جزئي .

- **Visitor** ونضيف هذا المتفاعل عندما يكون المتفاعل هو شخص لا يحتاج إلى تسجيل دخول لتنفيذ وظيفة ما على سبيل المثال .
- **Locater** من الممكن للـ locater تفعيل وظيفة في النظام، مثل أن يقوم النظام بتحديد موقع سيارة فإن سلك السائق طريق مسدود على سبيل المثال يقوم الـ locater بتنفيذ وظيفة " حالة استخدام " تنبيه السائق .
- **Alarm** : نستطيع معرفة إن كان المنبه يوجد ك متفاعل في النظام بطرح السؤال التالي : هل يوجد وظيفة في النظام تتم بشكل أوتوماتيكي عند حدوث أمر ما أو تغير حالة معينة، وبالتالي يتم تنبيه مستخدم النظام بوقوعها؟ مثل تنبيه عند زيادة ربح شركة ما فوق معدل محدد، أو نقصان أو خسارة، حدوث عملية تحويل لمبلغ من حساب مشبوه في نظام البنك .

Big shots are only little shots
who keep shooting





أنواع الـ Actors

Secondary Actor : هو نظام آخر مساعد للنظام الحالي وقد يكون API، يتلقى الرسائل ويجب عليها، فعندما يطلب النظام معلومات من الـ actor يكون المتفاعل في هذه الحالة عبارة عن secondary actor ، نفرقه عن الـ Primary من خلال اسمه ضمن المخطط ، لأن الـ secondary أحياناً يكون نظام.

Primary Actor : يقوم بعمل تهيئة "initiate" لوظيفة معينة، فهو أول من يرسل رسائل للنظام، حيث يقوم بتفعيل مهام النظام .

- ملاحظة : بعض المراجع تصنف Business actor وهو المتفاعل الذي يقوم بتفعيل وظيفة معينة بشكل غير مباشر " أي لا يتعامل مع النظام " ولا نمثله في مخططات الـ UML .
- للتوضيح : إذا أراد زبون شراء مادة ما، لكنه سيقوم بالشراء دون التعامل مع النظام، بل المحاسب هو من سيتعامل مع النظام، عندها نعد الزبون business actor لأنه كان سبب قيام المحاسب بتفعيل وظيفة معينة، والمحاسب هو System actor وعلى وجه الدقة يكون المحاسب عبارة عن Primary actor .

مثال : نظام الحكومة الإلكترونية ينفي عمل المواطن كمراسل " من يحضر الأوراق المطلوبة لعمل ما " بل تتعامل مع أنظمة أخرى " مثل نظام وزارة العدل ونظام وزارة التعليم العالي " لجلب المعلومات المطلوبة وبالتالي الأنظمة التي تتعامل مع الحكومة الإلكترونية تعد Secondary actor بالنسبة لنظام الحكومة الإلكترونية .

والـ actor يمكن أن يكون Primary في مكان ما و أن يكون Secondary في مكان آخر ومن أجل كل Actor يجب توصيفه ضمن جدول حالات الاستخدام "use case specification" كالتالي :

Name	اسم الـ Actor ويوضع بداخله .
Type	"primary or secondary"
Description	المهام التي يطبقها " صلاحيات "

- ملاحظة : وظائف الـ Actor تكون مختلفة فيما بينها حسب الـ role لا primary actor .
- تذكرة: لا نقوم بوضع الـ System " النظام " كـ actor

مثال : في نظام أتمتة جامعة افتراضية "on line" لدينا معلم يقوم بإلقاء المحاضرة وطالب يقوم بحضور هذه المحاضرة.

➤ من منهم يعتبر Primary actor ومن يعتبر secondary actor ؟

✓ كلاهما primary actor وكل واحد منهما لديه وظيفة يقوم بها "role" فوظيفة المعلم هي إعطاء

المحاضرة ويوجد فيها عدد من الخطوات وكذلك الأمر بالنسبة للطالب فوظيفته هي حضور المحاضرة وفيها عدد من الخطوات .

■ مثال : لدينا نظام للموظفين يقوم كل بداية شهر بإرسال كشف راتب لكل موظف .

بدايةً يجب علينا تحديد محدّدات النظام :

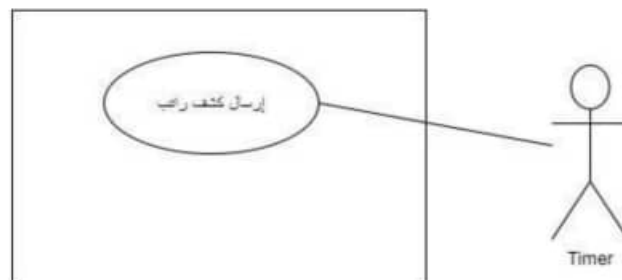
1. Use case : مثلاً نقوم بكتابة (إرسال كشف راتب) .

2. Actor : لمعرفة من هو الـ Actor هنا نقوم بطرح سؤال :

- من يقوم بإرسال هذا الكشف للموظفين ؟؟
- من نص السؤال نلاحظ أن النظام هو من يقوم بإرسال الكشف .



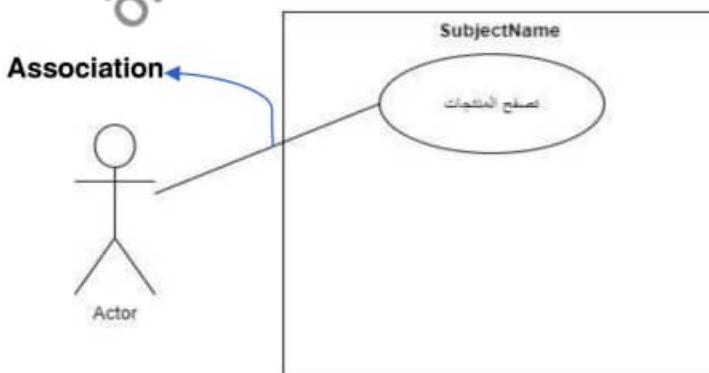
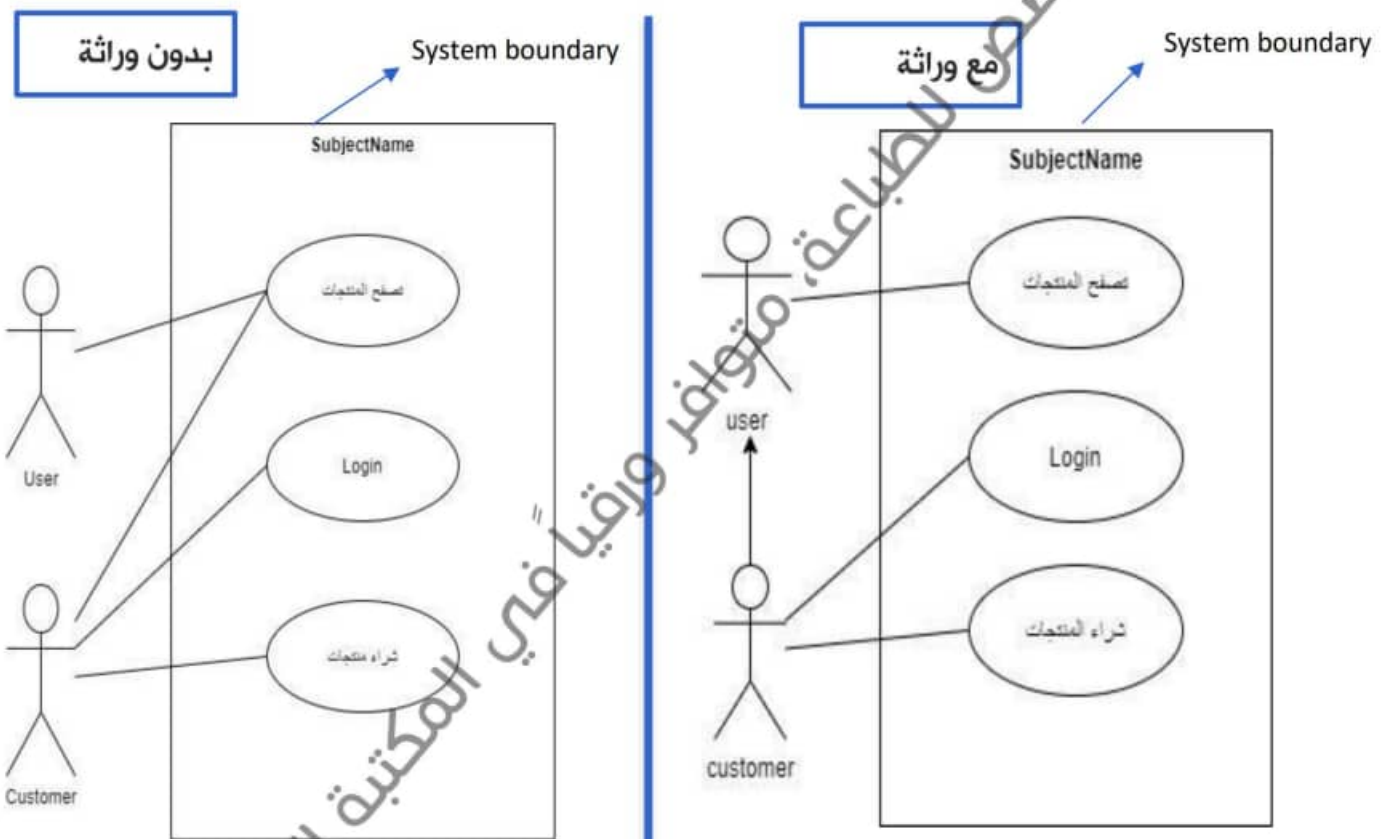
لا نقوم بوضع الـ system ك actor بل نقوم بوضع Timer "مؤقت أو مُجدول" .



Inheritance between Actors (الوراثة بين الفاعلين) :

لنفرض أنه لدينا نظام متجر إلكتروني، ولدينا Actor وليكن (User) والـ role الخاص به في النظام هو أنه يستطيع القيام بتصفح الموقع (المنتجات والعروض إلخ) "browsing" وذلك من دون تسجيل دخول .
ولدينا Actor ثاني هو الـ (customer) يستطيع تصفح الموقع والمنتجات بالتأكيد بالإضافة إلى تسجيل الدخول وشراء المنتجات .

من خلال وصف role كل Actor يتضح لنا أن الـ Customer هو User فعلياً مع بعض الصلاحيات الإضافية، أي أن الـ Customer يرث من الـ User (يستطيع الـ Customer القيام بجميع الوظائف التي يقوم بها الـ User)
ولكن العكس غير صحيح أي (الـ User لا يستطيع القيام بوظائف الـ Customer) .

العلاقات (Relationships) :

▪ علاقة الـ Association :

هي علاقة بين الـ Actor والـ Use Case

Relationships between Use Case

1. الكل مع الجزء Generalization : تستخدم هذه العلاقة عندما يتواجد ضمن حدود النظام وظيفة عامة لها وظائف فرعية، ويتم الإكمال ضمن وظيفة النظام بإحدى الوظائف الفرعية فقط "يمكن اختيار واحدة فقط من الوظائف الفرعية لإتمام الوظيفة العامة"، وبالتالي نلاحظ إمكانية تعميم حالات الاستخدام تماماً كما نقوم بتعميم الصفوف "classes" إذ ترث حالة الاستخدام الفرعية سلوك حالة الاستخدام العامة ومعناها، ويمكن أن تضيف إلى هذا السلوك أو تلغيه .
وتمثل هذه العلاقة ضمن مخطط Use case بسهم متجه من حالة الاستخدام الفرعية إلى حالة الاستخدام العامة .

مثال 1: يسمح نظام فواتير للمستخدم بدفع الفواتير للهاتف والكهرباء :



مثال 2: يسمح النظام للمستخدم بدفع الفواتير للهاتف أو الكهرباء باستخدام بطاقة بنكية أو Credit card :



2. Include : في هذه العلاقة، لا يمكن أن تتم حالة الاستخدام الأولى إلا بإتمام جميع الحالات المرتبطة بها بعلاقة include .

يرمز لهذا النوع من العلاقات بسهم منقط يتجه نحو الحالة المضمنة.

مثال : حتى يستطيع المستخدم أن يدفع فواتيره أو أن يحول رصيد، يجب أن تتم عمليات التحقق من هوية وصلاحيه رصيده :



3. Extend : هي علاقة تربط بين حالة استخدام وأخرى بحيث تعبر عن إمكانية إجراء الوظيفة الفرعية إنطلاقاً من الوظيفة الأساسية، أي قد يتم استدعاء الوظيفة الفرعية في علاقة الـ extend من قبل المستخدم أو لا .
مثال : يستطيع المستخدم استعراض تفاصيل كتاب انطلاقاً من وظيفة استعراض كتب :



جدول حالات الاستخدام

Use case ID	وهو الرقم المميز للـ Use Case ويميز حالة استخدام عن حالة أخرى
Use Case Name	وهو اسم الـ Use Case وهو الشكل البيضوي الذي يتم فيه تحديد المهام
Actors	وهم المتفاعلون مع حالة الاستخدام
Description	وهو وصف مختصر لحالة الاستخدام
Preconditions	الشروط الواجب توافرها قبل استخدام حالة الاستخدام (قبل حدوث حالة استخدام ما) وهي الحالات التي يجب على الـ User تفعيلها قبل استخدام الـ Use case، مثال: قبل توفر إمكانية البحث في كتاب يجب أن يوجد شرط استباقي يبين فيما إذا كان الكتاب موجود أم لا . بحث في الكتاب ← رسالة تبين إذا كان هذا الكتاب موجود أو لا
Postconditions	وهي الشروط التي يجب أن تتم بعد إتمام الوظيفة (حالة الاستخدام)
Flow of events(happy scenario) normal flow	مجموعة الخطوات التي ستتم لتحقيق الوظيفة المحددة للوصول إلى النتيجة المطلوبة، أي هو السيناريو الذي يتم لتحقيق الـ use case بشكل مفصل
Alternative flows	مجموعة الخطوات البديلة للـ normal flow فهو سيناريو يحقق الوظيفة أيضاً ولكن بطريقة مختلفة عن الـ normal flow (ولا نكتب فيه عدم تحقق حالة معينة بالتالي عدم القدرة على تحقيق الوظيفة).
Exceptions	الأحداث التي تؤدي إلى عدم تحقق الوظيفة وبالتالي الخروج من النظام أو تلغي مهمة أو تعود للمرحلة السابقة (أي عمل قد يعترضني في تحقيق الوظيفة). مثال: انتهاء صلاحية بطاقة الإنتمان ← لا يمكنني استخدامها .
Includes	حالات الاستخدام التي ترتبط معهم حالة الاستخدام الحالية بعلاقة include مثلاً : Include UC ID 2
Special Requirements	المتطلبات الغير وظيفية
Assumptions	وهي الافتراضات التي يملكها المستخدم حتى يستطيع التعامل مع الوظيفة (مواصفات أو مهارات) مثال : عمره يزيد عن 15 سنة أو يجيد اللغة الإنكليزية أو يستطيع التعامل مع كذا.

ملاحظات :

1. عندما يكون عدد حالات الاستخدام والمتفاعلين كبير نقوم بإنشاء subsystems داخل النظام ذلك بتجميع حالات الاستخدام المرتبطة مع بعضها البعض ضمن نظام جزئي .
2. في حال وجود وظيفة أو مجموعة من الوظائف التي يجب أن تتم قبل وظيفة ما (كالحاجة لإجراء تسجيل دخول قبل حجز كتاب) عندها نذكر ذلك في جدول سنذكره لاحقاً تحت مسمى preconditions (شروط مسبقة) كما يوجد أيضاً postconditions (شروط لاحقة) .
3. كل use case ليس لها actor فهي غير حقيقة ويمكن حذفها لأن كل فعل فيزيائي غير مرتبط بالنظام هو غير مقبول .
4. كل Actor ليس له use case فهو غير حقيقي ويمكن حذفه لأن كل فعل فيزيائي غير مرتبط بالنظام هو غير مقبول .

نهاية المحاضرة .

منصة للطباعة متوافر ورقياً في المكتبة المعتمدة